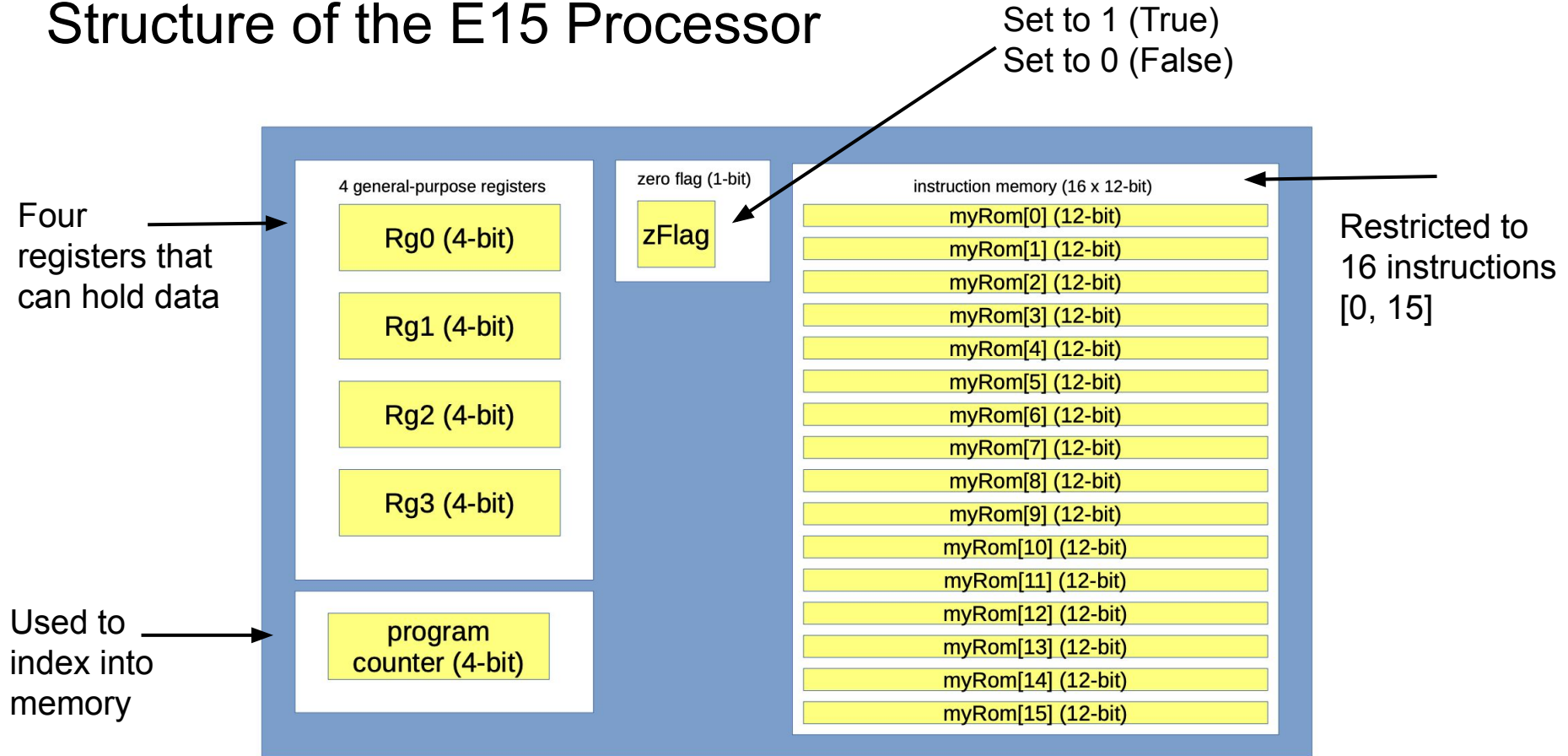


Recitation 3: E15

CS-UY 2214

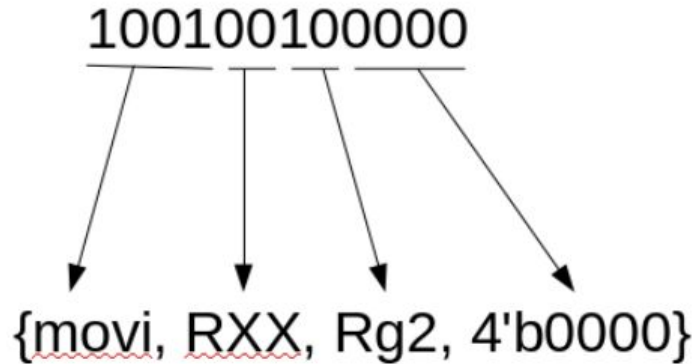
Structure of the E15 Processor



Instruction Format

Format: {**OPCODE**, **SRC**, **DST**, **IMM**}

Note that RXX = 2'b00 (same as Rg0)
Used when we don't have to modify a Rg



4-bit opcode

2-bit source
register

2-bit
destination
register

4-bit immediate
value

Opcodes

Each instruction as an unique opcode

<code>jmp</code>	<code>= 4'b0000,</code>	jump
<code>jz</code>	<code>= 4'b0010,</code>	jump if zero
<code>jnz</code>	<code>= 4'b0011,</code>	jump if not zero
<code>movi</code>	<code>= 4'b1001,</code>	move immediate
<code>mov</code>	<code>= 4'b1000,</code>	move
<code>addi</code>	<code>= 4'b1011,</code>	add immediate
<code>add</code>	<code>= 4'b1010,</code>	add
<code>subi</code>	<code>= 4'b1101,</code>	subtract immediate
<code>sub</code>	<code>= 4'b1100,</code>	subtract
<code>cmpi</code>	<code>= 4'b1111,</code>	compare immediate
<code>cmp</code>	<code>= 4'b1110;</code>	compare

Instruction Set

These five instructions do not modify the zFlag

`{jmp, RXX, RXX, imm};` → **PC = PC + IMM**
jump to the instruction at address pc+imm

`{jz, RXX, RXX, imm};` → **PC = (ZF == 1) ? PC + IMM : PC + 1**
jump to the instruction at address pc+imm if the zero flag is set

`{jnz, RXX, RXX, imm};` → **PC = (ZF == 0) ? PC + IMM : PC + 1**
jump to the instruction at address pc+imm if the zero flag is not set

`{movi, RXX, dst, imm};` → **REG[DST] = IMM**
store the value imm into the register dst

`{mov, src, dst, 4'b0000};` → **REG[DST] = REG[SRC]**
store the value in register src into the register dst

Instruction Set

These six instructions modifies the zFlag

`{addi, RXX, dst, imm};` → **REG[DST] = REG[DST] + IMM**

add the value imm to the value in register dst, store the result in dst

`{add, src, dst, 4'b0000};` → **REG[DST] = REG[DST] + REG[SRC]**

add the value in register src to the value in register dst, store the result in dst

`{subi, RXX, dst, imm};` → **REG[DST] = REG[DST] - IMM**

subtract the value imm from the register dst, store the result in dst

`{sub, src, dst, 4'b0000};` → **REG[DST] = REG[DST] - REG[SRC]**

subtract the value in register src from the value in register dst

`{cmpi, RXX, dst, imm};` → **ZF = (REG[DST] - IMM == 0) ? 1 : 0**

compare the value imm to the value in register dst; set the zero flag to 1 if they are equal, to 0 otherwise

`{cmp, src, dst, 4'b0000};` → **ZF = (REG[DST] - REG[SRC] == 0) ? 1 : 0**

compare the value in register src to the value in register dst; set the zero flag to 1 if they are equal, to 0 otherwise

zFlag

- All arithmetic instructions (add, addi, sub, subi, cmp, cmpi) set the zero flag on the basis of the ALU output: if the result is zero, the zero flag is set to one; otherwise, it is set to zero.
- In the case of cmp and cmpi, it modifies the zFlag because it performs [DST - SRC] or [DST - IMM] and sets the Rg to 1 if the difference is 0. However, no registers are modified.

E15 Example

- You must use {jmp, RXX, RXX, 4'b0000} to end the program
- If all 16 lines are not used, you must fill them with the above instruction

```
/*          OPCODE SRC  DST  IMMDATA */
myROM[0] = {movi, RXX, Rg1, 4'b0011};
myROM[1] = {add,  Rg1, Rg1, 4'b0000};
myROM[2] = {add,  Rg1, Rg1, 4'b0000};
myROM[3] = {movi, RXX, Rg2, 4'b1100};
myROM[4] = {cmp,  Rg1, Rg2, 4'b0000};
myROM[5] = {jz,   RXX, RXX, 4'b0011};
myROM[6] = {movi, RXX, Rg2, 4'b0001};
myROM[7] = {jmp,  RXX, RXX, 4'b0000};
myROM[8] = {movi, RXX, Rg2, 4'b0000};
myROM[9] = {jmp,  RXX, RXX, 4'b0000};
myROM[10] = {jmp,  RXX, RXX, 4'b0000};
myROM[11] = {jmp,  RXX, RXX, 4'b0000};
myROM[12] = {jmp,  RXX, RXX, 4'b0000};
myROM[13] = {jmp,  RXX, RXX, 4'b0000};
myROM[14] = {jmp,  RXX, RXX, 4'b0000};
myROM[15] = {jmp,  RXX, RXX, 4'b0000};
```


E15 Processor Diagram

E15 Processor

